aScalable Chat Platform

Custom Protocol Server Behind a Software Load Balancer

Design Approach & Technology Stack

---

1. Overview

This document records the approach chosen for each requirement in the problem statement, along with the specific tools and technologies selected, and the reasoning behind each decision. It is intended as a shared reference for the team and as supporting material for the project presentation.

2. Technology Stack

The following technologies were selected for the entire project:

- **Language — Go (1.21+):** Chosen for its native concurrency model (goroutines), which maps naturally onto handling many simultaneous long-lived socket connections — a core requirement of this project.
- **Core packages:** Standard library only: net, encoding/binary, encoding/json, net/http, sync, io. No external frameworks are required, keeping the system easy to reason about and free of unnecessary dependencies.
- **Git + GitHub:** Used for version control and as the required project deliverable repository.
- **Editor — VS Code:** Used for editing and running the project, with the Go extension for inline diagnostics and formatting.

Explicitly not used: Docker, Redis, a database, or any existing load balancer such as nginx/HAProxy. These were intentionally excluded — the problem statement requires the load balancer to be self-built, and the project scope does not call for persistence or containerization.

3. Part A — Protocol & Chat Server

Requirement	Approach	Tools / Tech	Reason
Custom protocol over TCP	Raw TCP sockets, custom application-layer protocol.	Go net package	TCP was chosen over UDP because the requirement calls for reliable, ordered, non-corrupted delivery, which TCP provides natively. Using UDP would mean rebuilding these guarantees manually, which is unnecessary complexity for a PoC.
Message format / framing	Length-prefixed frames: a 4-byte big-endian length header followed by a JSON payload.	encoding/binary, encoding/json	TCP delivers a byte stream, not discrete messages, so a receiver cannot know where one message ends and the next begins without an explicit boundary. A length prefix solves this cleanly and avoids the risks of delimiter-based framing (e.g. a delimiter character appearing inside message content).

Requirement	Approach	Tools / Tech	Reason
Protocol documentation	A separate PROTOCOL.md file documenting frame layout, message types, and design rationale.	Markdown	Keeping the spec in its own file (rather than only in code comments) makes it a reviewable, presentable artifact, and is an explicit deliverable in the problem statement.
Registration, join, broadcast, online list	An in-memory map of connected clients on the server; broadcast loops over the map and writes to every client except the sender.	Go map + sync.Mutex	A mutex is required because multiple client goroutines will read/write the shared client map concurrently; without it, the map access would race and could crash or corrupt state.
Reliable delivery	Rely on TCP's built-in ordering and retransmission; ensure the custom framing code never breaks that guarantee.	TCP (built-in)	TCP already solves reliability at the transport layer. The only risk of message loss or corruption at the application layer would come from incorrect framing code, so this requirement is met by disciplined reuse of one correct read/write implementation everywhere.
Partial read/write handling	Loop-read exactly N bytes for both the length header and the payload before treating a message as complete.	io.ReadFull	A single socket read is not guaranteed to return a complete message; io.ReadFull loops internally until the exact byte count is received or an error occurs, which directly satisfies this requirement.
Reconnection handling	Each client generates and persists a random session ID; it is resent on every JOIN so the server can re-associate a reconnecting client with its existing identity instead of treating it as new.	crypto/rand or a UUID library, local file for persistence	A session identifier is the simplest reliable way to distinguish 'this is the same client reconnecting' from 'this is a new client', without requiring any authentication system.

4. Part B — Software Load Balancer

Requirement	Approach	Tools / Tech	Reason
Run N server instances	The same server binary run multiple times with a different port flag (e.g. -port 9001, 9002, 9003).	Go flag package	Avoids code duplication — one binary, parameterized by port, is simpler to maintain and matches the requirement to run multiple instances of the same server.

Requirement	Approach	Tools / Tech	Reason
Custom load balancer	A separate Go program that accepts client TCP connections and proxies raw bytes to a chosen backend using bidirectional copying.	net.Listen, net.Dial, io.Copy	The problem statement explicitly disallows using an existing load balancer (nginx/HAProxy) — the exercise is to understand and build this mechanism directly. Piping raw bytes (rather than parsing the protocol inside the load balancer) also keeps the load balancer simple and decoupled from chat logic.
Two or more balancing strategies	A common Strategy interface with two implementations: round-robin and least-connections, selectable at runtime.	Go interfaces, atomic counters	An interface allows both strategies to be swapped without changing the surrounding proxy code, and satisfies the requirement for the active strategy to be switchable live during the demo.
Health checks	A background loop that attempts a TCP connection to each backend every few seconds and records whether it succeeded.	net.DialTimeout, goroutine + time.Sleep	A lightweight periodic probe is sufficient to detect an unresponsive backend without adding load, and matches the requirement for periodic (not per-request) health checking.
Automatic stop/resume routing	The active strategy filters to only backends currently marked alive before choosing one.	Shared state flag per backend, protected by a mutex or atomic value	Because the health check loop continuously updates the alive/dead flag, routing automatically resumes to a recovered backend with no additional recovery logic required.
Metrics view	A small HTTP endpoint inside the load balancer returning JSON (connections per backend, messages routed, alive status, active strategy), polled by a static HTML page.	net/http, HTML + JavaScript fetch()	Polling every 1–2 seconds is simple to implement and fully sufficient for a PoC; it avoids the added complexity of WebSockets while still satisfying the 'live metrics view' requirement.

5. The Hard Part — Sticky Sessions & Cross-Backend Messaging

Requirement	Approach	Tools / Tech	Reason
Sticky sessions / connection affinity	The backend is chosen once, at the moment the load balancer accepts a client connection, and the same	Falls out naturally from the proxy design (net.Dial once per accepted connection)	Because the load balancer proxies raw bytes over a single persistent connection pair, there is no per-request re-routing decision to make

Requirement	Approach	Tools / Tech	Reason
	backend is used for the lifetime of that connection.		— affinity is a natural consequence of the architecture rather than something requiring extra bookkeeping.
Cross-backend message visibility	A dedicated relay server. Each chat backend connects to it on startup; when a backend broadcasts a message locally, it also forwards it to the relay, which fans it out to all other backends.	A separate Go TCP server, reusing the same protocol/framing package	Two options were considered: a shared relay/bus, or having the load balancer itself fan out messages. The relay approach was chosen because it keeps the load balancer thin and protocol-agnostic (it only proxies bytes), and keeps all chat logic contained within the backend servers — a cleaner separation of concerns than making the load balancer protocol-aware.

6. Deliverables & How They Are Produced

Requirement	Approach	Tools / Tech	Reason
Git repository + README	Incremental commits per component, one shared repository with clear run instructions for each part.	Git, GitHub	Incremental commits (rather than one large commit at the end) demonstrate real progress and make integration issues easier to trace.
Protocol specification document	PROTOCOL.md, written and finalized early so all team members build against the same contract.	Markdown	Writing the spec before implementation prevents the server, load balancer, and relay from diverging on message field names or framing assumptions.
Live demo	A rehearsed, scripted sequence: start all components, run a load test of simulated clients, show cross-backend delivery, kill a backend mid-demo, switch strategies live.	A load-test script spawning concurrent simulated clients	The problem statement requires the demo to show distribution, failover, and a live strategy switch — rehearsing an exact command sequence in advance reduces the risk of a live failure during presentation.
Presentation	Structured around the problem, protocol design, architecture diagram, the sticky-session/cross-backend design decision, and limitations/next steps.	Slides	Matches the evaluation criteria directly — communication and the ability to defend design trade-offs are explicitly graded.

7. Explicit Scope Boundaries

In scope

- Running locally on one machine using multiple ports (or a few VMs/containers if convenient).
- A few dozen simulated clients — sufficient to demonstrate load distribution and failover.
- Text messaging only.

Out of scope (documented as future work, not implemented)

- TLS/encryption of the custom protocol.
- Persistent message history in a database.
- Chat rooms/channels (all clients share a single global broadcast scope).
- Full offline message queuing (a small in-memory replay buffer for brief reconnects is included; full queuing for extended offline periods is a stretch goal).
- File transfer, voice, video, and authentication beyond a plain username.

These exclusions are deliberate scope decisions made to keep the PoC focused on the core distributed-systems problems — protocol design, reliable framing, and stateful load balancing — rather than feature breadth.

8. Summary Table

Requirement	Approach	Tools / Tech	Reason
Protocol	Length-prefixed JSON frames over TCP	Go net, encoding/json, encoding/binary	Simple, debuggable, and solves the stream-framing problem directly.
Chat server	In-memory client registry, mutex-protected	Go map, sync.Mutex	Sufficient for PoC scale; avoids the complexity of external state stores.
Reconnection	Persistent client-side session ID	crypto/rand / UUID	Lets the server distinguish a reconnect from a new client without full auth.
Load balancer	Custom raw-TCP proxy	net, io.Copy	Required to be self-built; byte-level piping keeps it protocol-agnostic.
Strategies	Round-robin + least-connections	Go interfaces	Satisfies the two-strategy requirement with a clean, swappable design.
Health checks	Periodic TCP probe per backend	net.DialTimeout	Lightweight, sufficient to detect and recover from backend failure.
Cross-backend messaging	Dedicated relay server	Custom Go TCP server	Keeps load balancer thin; keeps chat logic in backend servers only.
Metrics	JSON API polled by a static page	net/http, HTML/JS	Simple, sufficient, avoids WebSocket complexity for a PoC.